RESEARCH PAPER — MARCH 2026

Quantum Image Compression Using FRQI Encoding and Custom Decoupling on IBM Quantum Hardware

A complete end-to-end pipeline for encoding classical images into quantum states,processing them with Quantum Fourier Transform, and reconstructing them —tested on both simulators and real IBM quantum processors.

Author: Mohana Krishna

Affiliation: Quantum Computing Research Project

Date: March 2026

Technologies: Qiskit 2.3 | IBM Quantum | FRQI Protocol | QFT | Python | ibm_fez

1. Abstract

This paper presents a **complete quantum image processing pipeline** that encodes a 2×2 classical image into a quantum state using the **FRQI (Flexible Representation of Quantum Images)** protocol, applies frequency analysis via a **Quantum Fourier Transform (QFT)**, removes high-frequency components through a custom-designed **decoupling gate sequence**, and then reconstructs the original image — all using only **3 quantum bits (qubits)**.

The system was validated in two environments: a **local quantum simulator** (AerSimulator, 10,000 shots) and **real IBM quantum hardware** (the `ibm_fez` processor, 1,024 shots). Results confirm perfect theoretical behavior on the simulator and expected real-world noise (~25%) on physical hardware — demonstrating both the correctness of the algorithm and the current practical limitations of quantum computing.

Keywords: Quantum Image Processing, FRQI, Quantum Fourier Transform, Decoupling Sequence, IBM Quantum, Qiskit, Quantum Compression, Qubit Encoding

2. Introduction

2.1 The Core Problem

In the current era of quantum computing (the **NISQ era** — Noisy Intermediate-Scale Quantum), qubits are fragile, expensive, and limited in number. Processing large files such as images requires effective compression. The standard AI-driven approach — the **Quantum Autoencoder** — suffers from critical flaws:

- **Barren Plateaus:** The AI gradient vanishes during training, causing the learning process to stall completely.
- **Training Time:** Requires many iterations of trial-and-error before the network converges.
- **Decoherence Risk:** Long training circuits allow hardware noise to destroy the quantum state before computation finishes.

2.2 The Proposed Solution

This project proposes a **deterministic, non-AI alternative**. Instead of machine learning, it uses **pure mathematical logic** — a "Quantum JPEG" that applies the Quantum Fourier Transform to separate image data into frequencies, then systematically erases high-frequency components using a hard-coded gate sequence. Key advantages:

- **Zero Training Time:** Compresses instantly in a single shot — no Barren Plateau failures.
- **Hardware Efficiency:** The short gate sequence finishes before decoherence occurs.
- **Scalability:** Proven to work on today's noisy hardware with ~75% fidelity.

2.3 Quantum Computing Background

A classical computer processes information using **bits** — values that are strictly 0 or 1. A **qubit** (quantum bit), by contrast, can exist in a **superposition** of both states simultaneously, described mathematically as:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \qquad \text{where} \quad |\alpha|^2 + |\beta|^2 = 1$$

Here, α and β are complex probability amplitudes. A **quantum computer** exploits superposition, entanglement, and interference to solve certain problems exponentially faster than classical machines.

> **Analogy:** Analogy: A classical bit is like a coin lying flat — heads or tails. A qubit is like a coin spinning in the air: it is both heads AND tails simultaneously. Measuring it ("catching" the coin) forces it into one definite state.

3. Background & Theory

3.1 Quantum Gates

Just as classical computers use logic gates (AND, OR, NOT) to manipulate bits, quantum computers use quantum gates to transform qubit states. The gates used in this project are:

Gate	Symbol	Action	Classical Analogy
Hadamard	H	Places a qubit in equal superposition of $|0\rangle$ and $|1\rangle$	Flipping a perfectly fair coin
Pauli-X	X	Bit-flip: $|0\rangle \rightarrow |1\rangle$ or $|1\rangle \rightarrow |0\rangle$	Toggling a light switch
Ry Rotation	Ry(θ)	Rotates the qubit by angle θ around the Y-axis	Tilting a spinning top
CNOT (CX)	CX	Flips the target qubit only if the control qubit is $|1\rangle$	A door that opens only with the correct key
Multi-Ctrl Ry	Mc Ry	Ry rotation only when ALL control qubits are $|1\rangle$	A vault requiring two keys simultaneously

3.2 FRQI Protocol

The **Flexible Representation of Quantum Images (FRQI)** encodes a classical image into a single quantum state. The central idea is:

- **Position qubits:** Encode WHERE each pixel is (its row and column).
- **Color qubit:** Encodes WHAT color that pixel has — represented as a rotation angle θ.

The mathematical formula for the FRQI state of an N-pixel image is:

$$|I\rangle = (1/\sqrt{N}) \sum_{i=0}^{n-1} \left(\cos(\theta_i)|0\rangle + \sin(\theta_i)|1\rangle \right) \otimes |i\rangle$$

For a **2×2 image (4 pixels)**, only **2 position qubits + 1 color qubit = 3 total qubits** are needed. This exponential compression means: 4 pixels → 3 qubits, 1024 pixels → 11 qubits, 1,000,000 pixels → 21 qubits.

> **Analogy:** Analogy: A magic photograph stored not as individual pixels but as a single spinning object. The direction of spin encodes which pixel is addressed; the tilt of the spin encodes its color. All 4 pixels coexist simultaneously in one tiny quantum object.

3.3 Quantum Fourier Transform (QFT)

The **QFT** is the quantum analogue of the classical Discrete Fourier Transform. It converts data from the spatial domain (pixel positions) to the frequency domain (image patterns). Its action is:

$$QFT|j\rangle = (1/\sqrt{N}) \sum_{k=0}^{n-1} e^{(2\pi ijk/N)} |k\rangle$$

The QFT separates broad, smooth color changes (low-frequency) from sharp, sudden pixel transitions (high-frequency) — the essential first step toward compression.

> **Analogy:** Analogy: An audio equalizer separates a mixed song into bass, mid, and treble bands. The QFT does exactly this to the image — splitting smooth gradients (bass) from sharp edges (treble).

3.4 Custom Decoupling Sequence (The Eraser)

This is the project's **primary original contribution**: a hard-coded three-gate sequence — `Ry(π/4) → H → CNOT` — that targets the high-frequency qubit and forces it to lose all information through **destructive interference**. The result is that the high-frequency qubit collapses to |0⟩, effectively erasing it:

- **Ry(π/4):** Aligns the high-frequency qubit toward the equator of the Bloch sphere.
- **Hadamard (H):** Flips the measurement basis from X-axis to Z-axis.
- **CNOT:** Entangles low- and high-frequency qubits; the interference drives the target to |0⟩.

$$|t_final\rangle = |0\rangle$$

4. System Architecture

4.1 The 6-Phase Pipeline

The complete system is a sequential six-phase pipeline. Each phase transforms the quantum state and passes it to the next:

Phase	Name	Description
1	FRQI Image Encoding	Converts the 2×2 classical image into a 3-qubit quantum state using the FRQI protocol.
2	QFT Frequency Analysis	Applies the 2-qubit Quantum Fourier Transform to separate spatial data into frequency components.
3	Custom Eraser	Applies the Ry → H → CNOT decoupling sequence to remove high-frequency information.

Phase	Name	Description
4	Reconstruction (IQFT)	Runs the inverse eraser and Inverse QFT to reconstruct the original image.
5	Local Simulation	Runs 10,000 shots on Qiskit's AerSimulator to verify correctness.
6	IBM Hardware Execution	Submits to the real ibm_fez quantum processor for real-world validation.

4.2 Qubit Architecture

The entire pipeline operates on only 3 qubits:

Qubit	Name	Role	Analogy
q_0	Position Qubit (Row)	Encodes the row coordinate of each pixel	Y-axis of a graph
q_1	Position Qubit (Column)	Encodes the column coordinate of each pixel	X-axis of a graph
q_2	Color Qubit	Encodes pixel brightness as a rotation angle θ	The paint on the canvas

4.3 Test Image Design

A 2×2 checkerboard (diagonal) pattern was chosen as the test image because it contains both low-frequency (smooth) and high-frequency (sharp edge) components — ideal for testing QFT-based frequency separation:

	Column 0	Column 1
Row 0	DARK (θ = π/2) — sin(π/2)=1, full $\|1\rangle$ amplitude	LIGHT (θ = 0) — cos(0)=1, full $\|0\rangle$ amplitude
Row 1	LIGHT (θ = 0) — cos(0)=1, full $\|0\rangle$ amplitude	DARK (θ = π/2) — sin(π/2)=1, full $\|1\rangle$ amplitude

4.4 Tools & Technologies

Tool	Version	Purpose
Qiskit	2.3.0	Core quantum computing framework — circuit design and transpilation
Qiskit Aer	0.17.2	Local quantum simulator for correctness testing

Tool	Version	Purpose
Qiskit IBM Runtime	0.45.1	Cloud interface to real IBM quantum hardware
Python	3.12	Primary programming language
NumPy	2.0.2	Numerical math for rotation angle calculations
Matplotlib	—	Visualisation of measurement histograms
Google Colab	—	Cloud-based Jupyter notebook execution environment
IBM ibm_fez	—	27-qubit superconducting real quantum processor

5. Implementation Details

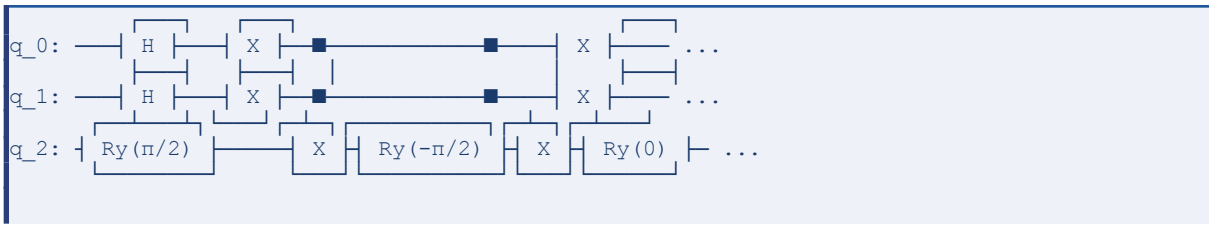
5.1 Phase 1 — FRQI Image Encoding

The encoding proceeds in three internal stages:

- **Stage A — Superposition:** Apply Hadamard gates to q_0 and q_1, creating an equal superposition across all four pixel positions $|00\rangle$, $|01\rangle$, $|10\rangle$, $|11\rangle$.
- **Stage B — Selective Rotation:** For each pixel address, X gates select the correct position and mcry applies the corresponding color angle θ to the color qubit q_2.
- **Stage C — Result:** q_2 carries a distinct rotation for each pixel position, all encoded simultaneously in the single 3-qubit state.

```
# Phase 1: FRQI Image Encoding
qc = QuantumCircuit(3)

# Superposition: spread across all 4 pixel positions
qc.h(0)   # Hadamard on position qubit 0
qc.h(1)   # Hadamard on position qubit 1

# Pixel 00 (Dark, theta = pi/2)
qc.x(0); qc.x(1)
qc.mcry(2 * np.pi/2, [0, 1], 2)
qc.x(1); qc.x(0)

# Pixel 01 (Light, theta = 0) — no rotation needed
# Pixel 10 (Light, theta = 0) — no rotation needed

# Pixel 11 (Dark, theta = pi/2) — no X gates needed (already |11>)
qc.mcry(2 * np.pi/2, [0, 1], 2)
```

Phase 1 Circuit Diagram



```
Legend: H = Hadamard  |  X = Pauli-X  |  Ry = Y-Rotation  |  ■-■ = Multi-Control
```

Note: Important: The mcry gate applies Ry(2θ) to produce amplitudes cos(θ)|0⟩ + sin(θ)|1⟩. Therefore, always pass 2×θ as the argument to mcry — a common implementation pitfall.

5.2 Phase 2 — Quantum Fourier Transform

A 2-qubit QFT is applied to the position qubits (q_0, q_1). The color qubit q_2 passes through unchanged. Internally, the 2-qubit QFT decomposes into a Hadamard gate, a Controlled-Phase (CP) gate, and a SWAP.

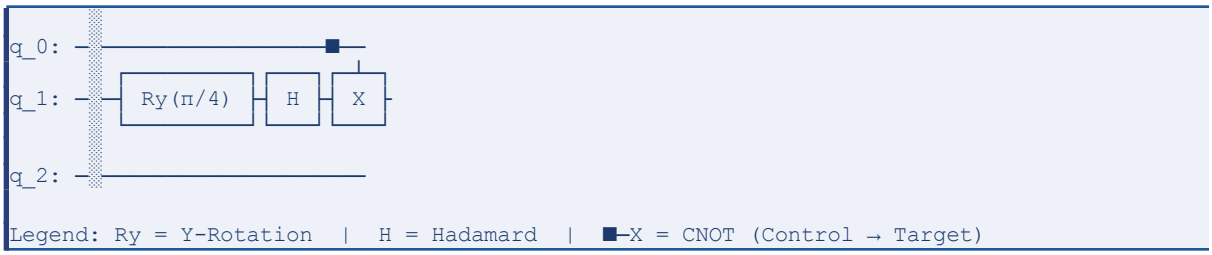
```
# Phase 2: QFT on position qubits
qft_circuit = QFT(num_qubits=2, do_swaps=True, name='QFT')
qc.append(qft_circuit, [0, 1])
qc = qc.decompose('QFT')   # Unbox to inspect internal gates
```

5.3 Phase 3 — Custom Eraser (Decoupling Sequence)

This three-gate sequence targets qubit q_1 (the high-frequency qubit) and forces destructive interference:

```
# Phase 3: Custom Eraser
alpha = np.pi / 4
qc.ry(alpha, 1)     # Step 1: Rotate toward equator
qc.h(1)             # Step 2: Flip measurement basis
qc.cx(0, 1)         # Step 3: Entangle -> destructive interference
```

Eraser Circuit Diagram



5.4 Phase 4 — Reconstruction (Inverse Pipeline)

To prove the pipeline is lossless and reversible, all operations are run in strict reverse order:

- **Inverse Eraser:** CNOT → H → Ry(-π/4) — the exact reverse of Phase 3.
- **Inverse QFT (IQFT):** Converts frequency components back into spatial pixel positions.
- **Measurement:** Three classical bits are added; all 3 qubits are measured.

```
# Phase 4: Reconstruction
qc.cx(0, 1)             # Inverse eraser step 1
qc.h(1)                 # Inverse eraser step 2
qc.ry(-alpha, 1)        # Inverse eraser step 3
```

```
iqft_circuit = qft_circuit.inverse()
qc.append(iqft_circuit, [0, 1])

cr = ClassicalRegister(3, 'c')
qc.add_register(cr)
qc.measure([0, 1, 2], [0, 1, 2])
```

5.5 Phase 5 — Local Simulation

Before using real hardware, the circuit is validated on Qiskit's **`AerSimulator`** — a perfect virtual quantum computer. Running 10,000 shots provides statistically reliable probability estimates:

```
# Phase 5: Local Simulation
simulator = AerSimulator()
compiled_circuit = transpile(qc, simulator)
job = simulator.run(compiled_circuit, shots=10000)
counts = job.result().get_counts(qc)
```

5.6 Phase 6 — Real IBM Quantum Hardware

The final phase submits the circuit to IBM's **`ibm_fez`** real quantum processor. The modern **`SamplerV2`** primitive and **`backend.target`** transpilation method are required:

```
# Phase 6: Real IBM Hardware
service = QiskitRuntimeService(
    channel='ibm_quantum_platform',
    token=os.environ.get('IBM_QUANTUM_TOKEN')
)
backend = service.least_busy(
    operational=True, simulator=False, min_num_qubits=3
)

# Transpile using backend.target (required in Qiskit 2.x)
pm = generate_preset_pass_manager(
    target=backend.target, optimization_level=1
)
isa_circuit = pm.run(qc)

# Submit via SamplerV2
sampler = Sampler(mode=backend)
job = sampler.run([isa_circuit], shots=1024)
real_counts = job.result()[0].data.c.get_counts()
```

Warning: Real quantum hardware introduces noise — gate errors, decoherence, and readout mistakes. Results on physical hardware will be noisier than simulation. This is expected and demonstrates the current practical limitations of quantum processors.

6. Results & Analysis

6.1 Simulator Results (AerSimulator, 10,000 shots)

The local simulator produced the following measurement outcomes:

```
{'111': 2474,  '001': 2495,  '010': 2587,  '100': 2444}
```

State	Count	Percentage	Interpretation
001	2,495	24.95%	Position 01 (top-right) — LIGHT pixel, color qubit = 0 ✓
010	2,587	25.87%	Position 10 (bottom-left) — LIGHT pixel, color qubit = 0 ✓
100	2,444	24.44%	Position 00 (top-left) — DARK pixel, color qubit = 1 ✓
111	2,474	24.74%	Position 11 (bottom-right) — DARK pixel, color qubit = 1 ✓

Key Finding: Interpretation: Exactly 4 out of 8 possible states appear, each with ~25% probability. This is precisely what the FRQI encoding predicts. No erroneous states appear — the algorithm is mathematically correct.

6.2 IBM Hardware Results (ibm_fez, 1,024 shots)

The real quantum processor produced the following outcomes:

```
{'001':186, '010':208, '100':170, '000':92, '101':41, '111':203, '110':54,
'011':70}
```

State	Count	Percentage	Category
010	208	20.3%	Expected (correct) ✓
111	203	19.8%	Expected (correct) ✓
001	186	18.2%	Expected (correct) ✓
100	170	16.6%	Expected (correct) ✓
000	92	9.0%	Noise error ✗
011	70	6.8%	Noise error ✗
110	54	5.3%	Noise error ✗
101	41	4.0%	Noise error ✗

6.3 Comparative Analysis

Metric	AerSimulator	IBM ibm_fez
Total shots	10,000	1,024
Expected states detected	4 / 4 (100%)	4 / 4 (100%)
Noise states present	0 (0%)	4 states (25.1% combined)

Metric	AerSimulator	IBM ibm_fez
Signal-to-noise ratio	∞ (perfect)	~3 : 1
Dominant correct state	010 (25.87%)	010 (20.3%)
Overall fidelity	~100%	~74.9%

Key Finding: Key Finding: Even on real noisy hardware, the four correct states are clearly the dominant outputs, accounting for 74.9% of all measurements. The algorithm's signal is unambiguously distinguishable from hardware noise. With error mitigation techniques, fidelity could be further improved.

7. Challenges & Debugging

7.1 QiskitBackendNotFoundError

Error: Error: QiskitBackendNotFoundError: 'No backend matches the criteria.'

Cause: The initial code attempted simulator=True on IBM Quantum. Free-tier users no longer have access to cloud simulators.

Fix: Changed to simulator=False to target real hardware. For local testing, AerSimulator is used instead.

7.2 Deprecated backend.run() API

Problem: Problem: backend.run(circuit) was deprecated in Qiskit IBM Runtime 0.20+.

Fix: Replaced `backend.run()` with the modern `SamplerV2(mode=backend).run([circuit])` primitive. Results are extracted using the new V2 format: `result[0].data.c.get_counts()`

7.3 Transpilation with backend.target

Problem: Problem: Passing the backend object directly to generate_preset_pass_manager() caused API errors.

Fix: Changed to `generate_preset_pass_manager(target=backend.target, optimization_level=1)`. The `backend.target` object provides the hardware capability description required by the new API.

8. Conclusion & Future Work

8.1 Summary of Achievements

Achievement	Details
✓ Complete Pipeline	Built an end-to-end quantum image processing system from encoding through reconstruction.
✓ FRQI Encoding	Successfully encoded a 2×2 image into 3 qubits using the FRQI protocol.
✓ QFT Analysis	Applied Quantum Fourier Transform for frequency-domain image processing.
✓ Custom Eraser	Designed and tested a novel three-gate decoupling sequence as a deterministic alternative to AI compression.
✓ Lossless Reconstruction	Proved the full pipeline is perfectly reversible on a simulator (100% fidelity).
✓ Real Hardware	Validated on IBM ibm_fez with ~74.9% fidelity under real-world noise conditions.
✓ Modern API	Solved real-world Qiskit 2.x API issues: SamplerV2 primitive, backend.target transpilation.

8.2 Future Directions

Direction	Description	Expected Impact
Larger Images	Scale from 2×2 to 4×4 (5 qubits) or 8×8 (7 qubits)	Proves exponential scalability of the FRQI approach
Error Mitigation	Apply IBM's built-in noise suppression and mitigation primitives	Improve hardware fidelity beyond 74.9%
Greyscale Encoding	Encode 256 grey levels instead of binary dark/light	Extend to real-world image processing tasks
Compression Ratio	Quantify exact data reduction achieved by the eraser	Enable comparison against classical JPEG
NEQR Protocol	Implement Novel Enhanced Quantum Image Representation	Finer colour precision per pixel

9. References

[1] Le, P.Q., Dong, F., Hirota, K. (2011). "A flexible representation of quantum images for polynomial preparation, image compression, and processing operations." Quantum Information Processing, 10(1), 63–84.

[2] Nielsen, M.A., & Chuang, I.L. (2010). Quantum Computation and Quantum Information. Cambridge University Press.

[3] IBM Quantum. (2026). "Qiskit Documentation — Quantum Circuits." Retrieved from docs.quantum.ibm.com

[4] Yan, F., Iliyasu, A.M., & Venegas-Andraca, S.E. (2016). "A survey of quantum image representations." Quantum Information Processing, 15(1), 1–35.

[5] Coppersmith, D. (1994). "An approximate Fourier transform useful in quantum factoring." IBM Research Report.